

Learning packet

RL for a lab-automation digital twin - five stops, in reading order

What this is

The five learning stops, in reading order, combined into one document. Track 4 first for motivation, then track 1 for the mechanics, then the demo, then the task, then the toolkit.

	Stop	Source	Lines	Assumes
1	Why — the door task and the research framing	<code>learn/4-door-task.md</code>	221	-
2	How — RL and TD3, plus a PyTorch-vs-TensorFlow appendix	<code>learn/1-rl-and-td3.md</code>	451	-
3	Run the demo — not in this packet	<code>learn/td3_pendulum_demo.py</code>	333	Stop 2
4	The task — Isaac Sim, Isaac Lab, and the existing <code>rl_games</code> door task	<code>learn/3-isaac-and-rl-games.md</code>	218	Stop 2
5	The codebase — finding your way around <code>jlab_opt_control</code>	<code>learn/2-toolkit.md</code>	181	Stop 2

The only hard rule is stop 2 before stops 4 and 5. Stop 1 stands alone and can be read first for motivation. If you would rather go strictly by track number, 4 -> 1 -> 2 -> 3 also satisfies every prerequisite.

Not included, and where to find it: the runnable demo (you are running it separately), `environment.md` if you need to get a machine going, and the three round files, which are provenance rather than learning material.

Where a track disagrees with a live file, the live file wins - and where it disagrees with gymnasium.farama.org, pytorch.org or the Isaac Lab docs, the docs win. Measured numbers and code defects are not repeated in the tracks; they live in `PUBLICATION_MATERIALS.md`, which is in the review packet.

Generated 2026-08-21 from `RL_DT@29f9573`. Assembled from the markdown sources named in each section header; nothing was rewritten for this packet. Chromium print does not support page numbers, so the section banners are the navigation.

How to use this packet

RL_DT/_docs/learn/README.md - 80 lines

The tracks, the prerequisite chain, and the honest gaps. The review list at the end of this section is the other packet.

Learning tracks

Four files. Read them in this order, or jump to the one you need — each says what it assumes. Everything here is **explanatory material**. Project decisions live in `../decisions.md`, measured numbers in `../PUBLICATION_MATERIALS.md`. Where a track disagrees with the official docs, the docs win.

The four tracks

	Track	What you get	Lines	Needs
[]	1 — RL and TD3	Enough RL to read and modify the ported agent: the Gymnasium contract, the Bellman target, TD3's three tricks. Plus a PyTorch-vs-TensorFlow appendix.	450	—
[]	<code>td3_pendulum_demo.py</code>	Run this. One self-contained file, TD3 solving Pendulum in ~50 episodes. The only runnable thing in <code>_docs/</code> .	333	Track 1
[]	2 — The toolkit	How to find your way around <code>jlab_opt_control</code> : registry + config, the layers, the reading order, where our code sits.	180	Track 1
[]	3 — Isaac and <code>rl_games</code>	What the twin is, the Isaac vocabulary, and a guided read of the door task that already exists.	218	Track 1
[]	4 — The door task	What we were actually asked to do, and the three ways it could become a paper.	221	—

Prerequisite chain: 1 → 2, and 1 → 3. Track 4 stands alone.

Reading order if you want the shortest path to being useful: 4 (why) → 1 (how) → run the demo → 3 (the actual task). Track 2 matters when you next touch toolkit code.

Reading order if you want motivation first: 4 → 1 → 2 → 3.

Track 1 is the heavy one at 450 lines because it replaces three separate files. Parts 1–2 are the sitting; Parts 3–4 and the appendix are lookup — don't try to absorb them in one pass.

Honest gaps

Stated so you don't go looking for material that isn't here.

- **PPO is not taught anywhere**, and it is the algorithm the Isaac Lab track actually uses. Track 3 reads its config; nothing explains the clipped surrogate objective or GAE.
 - **`rl_games` itself is undocumented** in this project. Track 3 names this as a real gap.
 - **SAC and DDPG** get a sentence each.
 - **Nothing in Track 3 has been run** — no RTX GPU on this machine. Its four "verify while reading" items are checks, not findings.
-

When you finish: the review list

Five files, in this order. This is the whole record of what has been done — no sixth file is needed, and none of the tracks above are on it.

- | | | |
|---|-----------------------------|--|
| [] 1. | ../../CLAUDE.md | § Current state, then § Roadmap |
| <hr/> | | |
| Where the project stands in ten lines, and what is planned. Start here so the rest has a frame. | | |
| [] 2. | ../PUBLICATION_MATERIALS.md | the findings index at the top, then the entries you care about |
| <hr/> | | |
| Every measured number and every code defect, with evidence attached. The most valuable file in the repo. Scan the index; read the entries that matter to you. | | |
| [] 3. | ../WORKLOG.md | the index table, then the round entries |
| <hr/> | | |
| What was done, round by round. Drill into ../rounds/ only when you need provenance for a specific claim. | | |
| [] 4. | ../strategy.md | Parts 2 and 5 |
| <hr/> | | |
| What was ported, what was reused, what was deliberately deferred, and the three architecture options with a recommendation. | | |
| [] 5. | ../team-discussion.md | all of it — it is short |
| <hr/> | | |
| What to present, what to ask and who can answer it, and the three fixes to propose upstream. The only file with an action attached. | | |

Two things to expect while reading. `PUBLICATION_MATERIALS.md` and `WORKLOG.md` overlap by design — the worklog says what was done, publication materials says what is worth telling someone else. And the fifth environment (`AdroitHandDoor-v1`) is marked as an **estimate awaiting a measured result** in exactly one place; if you see that marker, it has not been replaced yet.

Stop 1 of 5 — why

RL_DT/_docs/learn/4-door-task.md - 222 lines

Part 1 is a faithful conversion of the brief and does not go stale. Parts 2-4 are interpretation; anything marked [dated] was written before we read the code.

Track 4 — The door task: the assignment and the research framing

What you get: what we were actually asked to do, in the brief's own terms, and the three ways this work could become a paper. **Read it against:** `_reference/T2_RoboticProjectA.pdf` — the brief itself, which is the authority if anything here disagrees with it. **Size:** ~220 lines, but Part 1 is a two-page brief — one sitting. Read it **first** if you want motivation before mechanics. **Prerequisites:** none, though Track 3 makes Part 3 concrete.

Part 1 is a faithful conversion of the brief and does not go stale. Parts 2–4 are interpretation — mine and, in places, dated. Anything marked **[dated]** was written before we read the code.

Part 1 — The brief

"FAI RL for Robotics", Malachi Schram, June 2026.

Problem definition

Robotic automation in scientific lab spaces requires completing tasks in a **human-centric environment** — manipulating objects and equipment designed for **human hands**, not robotic grippers. The robot must handle **variable geometries**: slightly different approach angles, different manipulation orientations, a door handle that is rotated.

The overarching aim: **provide a generalized approach for using reinforcement learning in highly variable lab spaces using manipulators — mobile or stationary.** Framed as critical for lab-based automation.

Two example use cases:

1. **Open instrument doors with a mobile manipulator** to move labware between instruments. The doors are designed for humans; letting a robot open them enables automated sample runs without human intervention. Extends to cupboards and drawers in storage areas.
2. **Grasp falcon tubes and place them in a rack** for liquid-handling instruments. Must work from differing start positions and place tubes avoiding collisions, excessive torque and spills. Extension: multiple tubes in a specified order.

Use case #1 is what the twin is built for. Use case #2 shares the robot and is the natural follow-on — the asset already exists at `_reference/digital_twin_models-main/ReinforcementLearning/TubeRacking.usd`.

Environment

3D training scenes are **physically realistic**, built in Isaac Sim as USD with realistic gravity, collisions and articulations. Robots can be assembled in any orientation so **task randomization** is possible. Interacting equipment is customizable per task. End effectors can grip (**with tactile force sensors**), suction, or imitate a human hand. **RGB cameras and lidar** can be added. Instrumentation uses realistic scale, geometry and mass; geometry may be simplified for performance — with the caveat that **enough physical detail must remain that reward hacking cannot bypass physical constraints.**

State, action, reward — as specified

Observation components for a manipulator gripper: end-effector pose; joint angles and velocities; gripper width / grasp state; pose estimate of the target (6D door position and rotation); target pose or rack slot; **relative transforms** (target w.r.t. gripper, end goal w.r.t. gripper); **force and torque at the wrist** (UR has F/T estimation, an external sensor is better); contact indicators and collision flags.

Actions: (1) open the door hinge to x degrees using the handle — grab the handle, move the door along an arc; (2) place a falcon tube in a rack.

Rewards. Event bonuses: stable-grasp bonus after N steps, first contact with the slot rim, insertion achieved, successful release, and a large dominating full-success bonus. Penalties: drop, collision or self-collision, **force/torque penalty or constraint — the reward must not encourage brute-force pushing**, and excessive tilt.

Section 1.2, "Tasks", is blank in the brief. A placeholder never filled in. That is not an oversight to route around — it is the single largest piece of missing specification, and it is why success thresholds and the randomization envelope are still open questions.

Goal, verbatim in substance

To provide a **robust framework** for RL development for lab-based automation. A **modular test bed** to deploy RL algorithms and provide useful tools to scientists who need to increase data-collection capability.

Part 2 — What kind of project this is

A framework project first, a task project second. The stated goal is not "solve the glass door" — it is a modular, robust RL test bed for lab automation, with door-opening and tube-racking as the first two proving tasks. That framing should drive scoping, code structure, and the paper.

The generalization requirement is the real point. The brief stresses variable geometry and task randomization repeatedly. So the success criterion is not "open one door once" but **open the door across a randomized distribution of poses and angles**. Domain randomization and robust evaluation belong in from day one, not bolted on later. Measured against that bar, the existing task's `position_range: (0.0, 0.0)` reset is the largest gap in the project — see Track 3 Part 2.

Reward hacking is called out explicitly, twice. Consequences: reward shaping is a first-class engineering task; **success metrics must stay separate from training reward**; force/torque penalties and physical fidelity in the sim matter. All three are project rules in `CLAUDE.md` for this reason.

Who owns what. The brief is Malachi's; the twin is the DT developer's (`prat615`, project `LDRD_RoboticsAI`); the RL toolkit is Malachi's group's, written for accelerators. Our job is the middle: define the MDP, wrap the twin as a trainable env, train and evaluate. Alvika leads the RL work; Dewei supports.

The success ladder

A proposal, not an agreed plan — item 4 is where the brief's actual requirement starts.

1. **Reach** the handle, base fixed — proves the env, rewards and logging work at all.
2. **Grasp** the handle stably.
3. **Open** to a target angle from a fixed pose.
4. **Open** under randomized approach angle and handle rotation. ← the real bar

5. Add the **mobile base** (navigate, then manipulate) — only if in scope.

6. A **second instrument or the tube task** — what substantiates the word "framework".

The existing code aims at roughly step 3 with a heavily scripted reward, and commits to a full sequence (approach → grasp → 10° → 90° → 180° → release → return home). Whether 180° is the intended success criterion is an open question.

Part 3 — The paper

Three candidate theses

(A) Framework + first task. *"A digital-twin RL framework for human-centric lab automation, demonstrated on a mobile-manipulator door-opening task with generalization over variable geometry."* Matches the brief exactly; the contribution is a reusable pipeline plus a hard concrete task. Risk: framework papers still need a convincing task result, so the door has to work.

(B) Method paper — physics-informed / interpretable RL for contact-rich manipulation. *"Injecting known contact/constraint physics into RL for safe, sample-efficient, interpretable lab manipulation."* Continues the group's SINDy / LC-TD3 line (Colen 2026) into robotics; strongest novelty; natural MLST fit. Risk: highest — you must show the physics-informed component beats a strong baseline, so don't commit before the baseline works.

(C) Benchmark / empirical study. *"Benchmarking PPO/SAC/TD3, ± domain randomization, ± physics priors, on a lab door-opening digital twin."* Lowest risk, always publishable, a natural byproduct. Lower novelty alone.

Recommendation: build so that **(A) is the guaranteed paper, (C) is a section inside it, and (B) is the upside** you claim only if the result is strong. This makes the paper's ambition follow the results instead of betting up front. Aligning on this is the main thing to settle with Malachi.

Skeleton, framing A with the method angle built in

Working title: *A Physics-Informed Digital-Twin Reinforcement Learning Framework for Human-Centric Laboratory Automation: Mobile-Manipulator Door Opening.*

1. **Introduction** — lab automation needs robots operating human-designed equipment under variable geometry; the gap is that general RL frameworks aren't tailored to constrained, contact-rich, human-centric lab tasks. Contributions: (i) a modular digital-twin test bed; (ii) an MDP formulation with force/torque constraints and variable-geometry generalization; (iii) benchmarked baselines; (iv) *if it pans out* a physics-informed interpretable component.
2. **Related work** — RL for manipulation and articulated objects; sim-to-real and domain randomization; physics-informed and constrained RL; lab/science automation. Cite Rajput 2025 and Colen 2026 as the lineage this extends from accelerators into robotics.
3. **System and digital twin** — the Isaac Sim 5.1 scene, fidelity choices, and the explicit anti-reward-hacking rationale.
4. **Problem formulation** — the MDP as specified in Part 1, plus termination and success criteria and the randomization envelope that defines "generalization".
5. **Method** — algorithms on parallel Isaac Lab envs; the physics-informed component as an upside section; domain randomization and sim-to-real considerations.
6. **Experiments** — learning curves and **success rate over randomized poses** (the real metric); ablations on dense vs sparse reward, ± randomization, ± physics prior, algorithm choice; safety metrics (peak force/torque, collision

rate) showing no brute-force solution; interpretability evidence if (B); generalization vs randomization breadth; failure modes.

7. **Discussion** — what transfers to other instruments and tasks; limitations; the sim-to-real gap.

8. **Conclusion** — the test bed as reusable; tube-in-rack; hardware transfer; multi-step workflows.

What makes it publishable, and where the risk is

The strongest novelty lever is **physics-informed + interpretable** — the group's signature, and what separates this from a generic "RL opens a door" paper. Gate it on a working baseline. Second lever: rigorous **generalization over variable geometry with an explicitly force-constrained solution**, which answers the brief directly and is a clean story. Third: the **released test bed** itself.

Main risk: long-horizon contact-rich manipulation is genuinely hard, so let ambition follow results. And **reproducibility must be fixed up front** — the reference toolkit is non-reproducible by design, which is exactly why our own agent takes a `seed` `cfg` key and why the evaluation-seeding trap in `../PUBLICATION_MATERIALS.md` is worth knowing before you generate results rather than after.

Minimum results before writing

1. A working baseline that opens the door from a fixed pose — proves the pipeline.
2. A generalization result over randomized approach angle and handle rotation — the headline metric.
3. Constraint evidence: force and torque stayed within limits.
4. (*Upside*) the physics-informed variant beating the baseline on sample efficiency, safety or interpretability.
5. All of it with **fixed seeds, ≥ 3 seeds, reported variance**. Still outstanding — the current four-environment comparison is one seed per arm.

Part 4 — What has changed since this was written

[dated] The interpretation above was written in July 2026, before any code was read. Two things have moved:

- **The twin is not asset-only.** A working Isaac Lab door task with `rl_games` PPO already exists. Anything above that reads as "we must build the env from scratch" should be read as "we must adapt and fix the env that exists" — Track 3.
- **The agent side has landed.** A PyTorch TD3 now exists inside the toolkit, tested and matched against the Keras baseline on four Gymnasium environments. So the "port the algorithms" line in the ladder above is done, and what remains is vectorization and the Isaac integration — `../strategy.md`.

What has **not** changed: Section 1.2 of the brief is still blank, no success threshold or tolerance has been set, the randomization envelope is unspecified, the observation modality is undecided beyond "privileged sim state for now", and whether sim-to-real is in scope is unknown. Those are the decisions that most change what gets built, and they are the top of `../team-discussion.md`.

What this track does not cover

- **The algorithms** — Track 1.
- **The toolkit the method contribution lives in** — Track 2.
- **The scene and the existing task code** — Track 3.

- **Decisions already made and why** — `../decisions.md`.
- **The questions themselves, ranked and grouped by who can answer them** — `../team-discussion.md`.

Stop 2 of 5 — how

RL_DT/_docs/learn/1-rl-and-td3.md - 452 lines

The heavy one. Parts 1-2 are the sitting; Parts 3-4 and the appendix are lookup - do not try to absorb them in one pass.

Track 1 — Fundamental RL and TD3

What you get: enough RL to read and modify the ported agent — the Gymnasium contract, the Bellman target, and the three tricks that make TD3 work. **Read it against:** `td3_pendulum_demo.py` in this folder, running. **Size:** 450 lines, the longest track by a wide margin — it replaces three separate files. Parts 1–2 are the sitting; Parts 3–4 and the appendix are lookup, read them when you hit them. **Prerequisites:** none. Everything else in `learn/` assumes this one.

Written during Round-2026-08-10, updated after the port landed. **Explanatory notes, not project decisions** — those live in `../decisions.md`. Where this disagrees with `gymnasium.farama.org` or `pytorch.org`, the docs win.

How to run the demo

```
cd "<PROJECT_ROOT>/SciOptControlToolkit"          # any env with torch + gymnasium
.venv/Scripts/python "<REPO>/_docs/learn/td3_pendulum_demo.py"      # 60 episodes
.venv/Scripts/python "<REPO>/_docs/learn/td3_pendulum_demo.py" --episodes 30 --seed 1
```

A teaching artifact and a smoke test, not the port. Deliberately one file with hyperparameters as inline constants so the whole algorithm reads top to bottom. The shipped agent is `jlab_opt_control/agents/torch_td3.py` in the toolkit clone, with hyperparameters in `cfigs/torch_td3.cfg`, per `CLAUDE.md` ("no hyperparameters in code").

Part 1 — The environment side

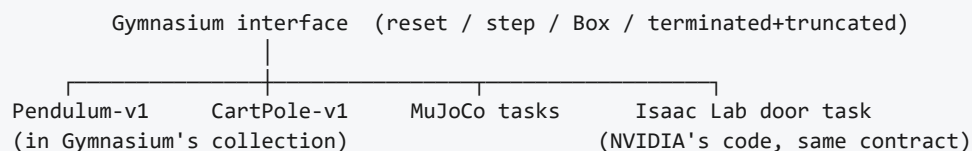
Gymnasium is a library; Pendulum-v1 is one environment in it

Gymnasium is the maintained fork of OpenAI Gym, run by the Farama Foundation since OpenAI stopped developing Gym around 2022. `import gymnasium as gym` is the modern import; a tutorial doing `import gym` is pre-2022 code.

It provides **two** things, and conflating them hides why this project cares:

1. **An interface specification** — `reset()` / `step()`, `Box` and `Discrete` spaces, the `terminated`/`truncated` split. Abstract; contains no physics.
2. **A bundled collection of environments** — `CartPole`, `Pendulum`, `MuJoCo`, `Atari`.

The collection is the obvious half. **The interface is the half that matters here**, because Isaac Lab implements #1 without being part of #2: `ManagerBasedRLEnv` is NVIDIA's code, in a different package, on a different physics engine, using the same contract.



Pendulum and the door task are siblings under one specification. That is why one agent can drive both, and why the agent layer must import without Isaac.

The core contract

`reset()` → `(observation, info)`. `step(action)` → `(observation, reward, terminated, truncated, info)`.
The resulting loop (`drivers/run_continuous.py`, `run_episode`):

```
state, _ = env.reset()
while not done:
    action, _ = agent.action(state)
    next_state, reward, terminate, truncate, _ = env.step(action)
    done = terminate or truncate
    agent.memory((state, action, reward, next_state, terminate))
    agent.train()
    state = next_state
```

Observe → act → receive → store → learn → repeat. The environment owns the task, the agent owns the learning, neither knows the other's internals.

terminated VS truncated — the one thing to really understand

Gymnasium's headline change from old Gym's single `done` flag. It exists because merging the two was causing silently wrong RL implementations.

Flag	Meaning	Example
terminated	Ended for a reason inside the MDP — a genuine terminal state, no future beyond here.	CartPole pole angle > 12°
truncated	Cut off from outside — time limit, external abort. The state was fine; we stopped watching.	hit the 200-step cap

Both end the loop, but they mean opposite things to the learner, because of value bootstrapping. The critic's Bellman target is `q_targets = rewards + gamma * target_q * (1.0 - done)`. When `done = 1` the future term vanishes — the state is worth exactly its immediate reward. **Correct for termination** (the pole fell; there is genuinely no future). **Wrong for truncation** (the pendulum at step 200 is still spinning and has plenty of future value). Zeroing the bootstrap there teaches the critic that good states are worthless, and the error propagates backwards through the value function.

Rule: **bootstrap through truncation; cut the bootstrap only on termination**. Note how the loop above uses `done` for control flow but stores `terminate` alone in the buffer. One careless `done` there is a silent bug that just makes learning quietly worse — which is why the ported agent has a dedicated test for it, and why that test was verified by breaking the line on purpose (`../PUBLICATION_MATERIALS.md` § Method choices, 2026-08-19).

Pendulum caveat: its `terminated` is *always* `False` — no failure state, no goal state, it just spins until truncation at 200 steps. So the stored flag is always 0 and **this code path is never exercised by the smoke test**. Do not read "Pendulum works" as "termination handling is correct." Of the five environments used in the port comparison, only `Hopper-v5` terminates for real. The door task will too.

Spaces

A `Space` describes the shape of observations and actions. Three matter:

- **Box** — continuous, bounded, n-dimensional. Has `.shape`, `.low`, `.high`, `.sample()`.
- **Discrete(n)** — one integer in `0..n-1`. TD3 emits real values through a `tanh`, so it structurally cannot handle this; hence the toolkit's `assert "Box" in str(type(env.action_space))`.

- **Dict** — a named bundle of sub-spaces, and its `.shape` is `None`, which breaks any code reading `observation_space.shape[0]`. All 217 goal-conditioned gymnasium-robotics environments use it; see `envs/robotics_flat.py` in the toolkit clone for the wrapper that opened them up.

Spaces are also how the agent self-configures: it reads `observation_space.shape[0]` and `action_space.shape[0]` to size networks, and `.high/ .low` to scale output. No dimension is hardcoded, which is what lets the same agent run Pendulum now and a 6-DOF arm later. `action_space.sample()` does real work too — during warm-up the agent ignores its network and samples uniformly, because an untrained network's actions are nearly constant and teach the critic almost nothing.

Pendulum-v1, and why –200

Swing-up: a rigid pendulum on a frictionless pivot, torque at the pivot, random start angle (usually hanging down). The torque limit is **deliberately too weak to lift it directly**, so the policy must learn to pump energy in and swing.

- **Observation — 3 numbers:** `cos(θ)`, `sin(θ)` in $[-1, 1]$, and `θ` in $[-8, 8]$. Why `cos/ sin` rather than the angle? Angle wraps: $\theta = +3.14$ and $\theta = -3.14$ are the same physical state at opposite ends of the numeric range, a discontinuity the network would have to learn. `(cos, sin)` is continuous everywhere. The trick recurs throughout robotics — we will want it for door-hinge angles.
- **Action — 1 number:** torque in $[-2, 2]$.
- **Reward — always negative:** $r = -(\theta^2 + 0.1 \cdot \dot{\theta}^2 + 0.001 \cdot \text{torque}^2)$, θ measured from upright. Penalties for being away from vertical (dominant), spinning fast, and spending effort. Best possible is 0, so returns are negative and "better" means "closer to zero."
- **Episode length:** exactly 200 steps, always.

Policy	Return	Reasoning
Do nothing	≈ -1900	hangs at bottom, $\theta \approx \pi \rightarrow \approx 9.9 \text{ cost/step} \times 200$
Random torque	≈ -1200	jiggles, occasionally swings up by luck
Good policy	≈ -150 to -200	swings up in ~ 30 – 50 steps, then holds near-zero cost
Theoretical max	0	unreachable — requires starting upright

Even a perfect policy cannot approach 0: the pendulum *starts* down and every step of the swing-up accrues cost, worth roughly -100 to -150 . So **–200 means solved**. The gap from -1200 to -200 is large and unambiguous, which is why Pendulum is the standard smoke test — a real bug lands you near random and you *know*, with no marginal result to interpret.

Because the start angle is random, single-episode returns are noisy. Judge the rolling average (the toolkit's `nepisode_avg`, default 20) over multiple seeds, never one episode.

Wrappers

An environment wrapping another, same interface, modified behaviour — composable like decorators. The toolkit uses three: `TimeLimit` (injects `truncated` after N steps; redundant on Pendulum, which already has a 200-step limit), `RescaleAction` (remaps the action range), and `FlattenObservation` (collapses a `Dict` into one flat vector — the one that unlocked the Fetch/Hand/Maze/Kitchen families; note it concatenates in **sorted key order**, so the agent's input layout depends on that ordering rather than on anything declared).

Wrappers are where environment-specific adaptation belongs, which keeps the agent generic. For Isaac, observation normalization should live here rather than inside TD3.

Seeding — three RNGs, not one

RNG	Seeded by	Controls
Environment	<code>env.reset(seed=...)</code>	starting states, env noise
Action space	<code>env.action_space.seed(...)</code>	<code>action_space.sample()</code> — the warm-up actions
Torch / NumPy	<code>torch.manual_seed</code> , <code>np.random.seed</code>	weight init, exploration noise, buffer sampling

Seeding the env does **not** seed the action space. The Keras agent never seeds it at all, so even with everything else fixed its warm-up differs run to run — and warm-up is thousands of transitions shaping the entire early buffer.

TorchTD3 seeds all three from a `seed` cfg key, before any model is constructed. Also: `reset(seed=...)` seeds the generator, so pass it **once** at the start; the same seed on every reset gives the identical starting state 200 times.

A subtler trap sits one step out: seeding *training* does not make *evaluation* reproducible, and the evaluation number is the one that gets published. Measured in this demo — 170 points of spread across four runs of the same seed. See `../PUBLICATION_MATERIALS.md` § Failure modes, 2026-08-12.

What Pendulum does not prove

- **Vectorization** — Isaac Lab returns 256+ envs as batched GPU tensors; this is one env returning CPU NumPy. Different loop shape, and replay buffers need real adaptation.
- **Termination handling** — always `False` here, as above.
- **Scale** — 3 obs / 1 action vs dozens / 7 joints. Hyperparameters will not transfer directly.
- **Anything task-shaped** — reward design, scene correctness, whether the door problem is well-posed.

Pendulum validates *the algorithm*. Everything else remains ahead.

Part 2 — TD3

What you should see

ep	1	warmup	steps	200	return	-1477.3	avg10	-1477.3		
ep	5	warmup	steps	1000	return	-1181.9	avg10	-1315.5	EVAL	-1215.4
ep	15	train	steps	3000	return	-893.0	avg10	-984.2	EVAL	-382.1
ep	30	train	steps	6000	return	-238.6	avg10	-251.0	EVAL	-155.2

Warm-up (ep 1–5) sits at −1200 to −1500 on uniform random actions with no learning; early learning (5–20) climbs −1200 → −400 as the critic becomes useful and the actor follows; convergence (20–50) reaches −400 → −200. `EVAL` is greedy and averaged over 3 episodes, so it is the honest number and usually **better** than the training return. Expect noise — TD3 sometimes plateaus near −700 for ten episodes and then drops fast. Still $\approx$ −1200 by episode 30 means something is wrong.

The algorithm in one page

TD3 = Twin Delayed DDPG. Two networks with different jobs:

- **Actor** $\pi(s) \rightarrow a$. A *deterministic* policy: one state in, one action out. No probability distribution (that's SAC). Job: output the action the critic likes best.

- **Critic** $Q(s, a) \rightarrow \text{scalar}$. Expected discounted return from taking a in s . Job: predict the Bellman target.

They bootstrap each other. The critic learns to predict returns from collected experience; the actor learns to pick actions the critic scores highly. Neither is given the answer — the only ground truth is the reward stream.

The learning signal. The value of a state-action pair is the reward now plus the discounted value of where you end up, $Q(s, a) \approx r + \gamma \cdot Q(s', \pi(s'))$. Turn it into a regression target:

```
target = rew + GAMMA * (1.0 - done) * target_q      # what Q *should* be
critic1_loss = F.mse_loss(self.critic1(obs, act), target)
```

$\gamma = 0.99$ means a reward 100 steps out is worth ≈ 0.37 of an immediate one — enough foresight to plan a swing-up, short enough to stay numerically stable.

Why DDPG needed fixing. DDPG is this loop with one critic, and it systematically **overestimates**. Q is a noisy estimate, the actor is trained to maximise it, so the actor is an error-seeking missile that finds wherever Q is spuriously high. That inflated value then enters its own target via $Q(s', \pi(s'))$, so the error compounds and training collapses. TD3's three tricks each attack one part of that loop.

Trick 1 — clipped double-Q

```
target_q = torch.min(self.critic1_target(next_obs, next_act),
                    self.critic2_target(next_obs, next_act))
```

Two independent critics, and the target uses the **minimum**. Both overestimate, but in *different places* (different random init, different gradient noise), so for the actor to exploit an error **both** must be wrong at the same point — far less likely. A deliberate downward bias traded for stability: underestimating a good action is survivable, overestimating a bad one is not. The critics stay independent — separate parameters, losses and optimizers. Only the `min` couples them.

Trick 2 — target policy smoothing

```
noise = (torch.randn_like(act) * self.action_scale * POLICY_NOISE).clamp(
    -NOISE_CLIP * self.action_scale, NOISE_CLIP * self.action_scale)
next_act = (self.actor_target(next_obs) + noise).clamp(self.low, self.high)
```

Similar actions should have similar values — physics is smooth, torque 1.00 and 1.02 cannot differ wildly in worth. But a neural net can develop a sharp spurious spike and the actor will drive straight into it. Sampling the target at $\pi(s') + \text{noise}$ averages Q over a small neighbourhood, smoothing narrow peaks away; it is a regulariser on the value landscape. The clip stops a rare large Gaussian draw from sampling a completely unrelated action.

This noise is on the **target action inside the critic update** — different from exploration noise, which is added to the action actually sent to the environment. Two noises, two jobs, easy to conflate on a first reading of TD3.

Trick 3 — delayed policy updates

```
if self.total_updates % POLICY_DELAY == 0:      # every 2nd critic update
    actor_loss = -self.critic1(obs, self.actor(obs)).mean()
```

The actor is only as good as the critic guiding it, so updating it every step means chasing a target that is itself thrashing. Every *other* step lets the critic settle between policy changes. Target networks are soft-updated inside the same branch, so they move on the actor's schedule.

The actor loss deserves a second look. There is no "correct action" label anywhere — no supervised signal. We want to *maximise* Q and optimizers minimise, hence the minus sign. Gradients flow backwards through `critic1` into the actor's weights: the critic is a differentiable model of "how good is this action," and the actor does gradient ascent through it. Only possible because actions are continuous and Q is differentiable with respect to them.

`critic1`'s parameters receive gradients here too — but `actor_opt` was constructed with only `self.actor.parameters()`, so `actor_opt.step()` cannot touch them, and they are cleared at the next `critic1_opt.zero_grad()`. Worth understanding rather than trusting: in PyTorch, *what gets updated is determined by which parameters you handed the optimizer*, not by where gradients flowed.

Target networks

Three networks became six: each has a slow-moving copy. Without them the critic's target depends on the critic's own current weights, so every update moves the target it is chasing — unstable in exactly the way hitting a target attached to your own hand is unstable.

```
def soft_update(net, target_net, tau):      # tau = 0.005
    for p, tp in zip(net.parameters(), target_net.parameters()):
        tp.mul_(1.0 - tau).add_(tau * p)    # target = 0.995*target + 0.005*online
```

Polyak averaging: each update pulls the target 0.5% toward the online net, tracking it ~200× slower, so it is effectively a stable reference. Targets are never trained — hence `p.requires_grad_(False)`.

Note the asymmetry in that line. At `tau = 0.5` the two coefficients are equal, so swapping them changes nothing, which is why a test at `tau = 0.5` cannot detect the swap and one at `tau = 0.25` can. That is a real finding from the port, not a hypothetical.

The replay buffer

```
idx = rng.integers(0, len(self), size=batch_size)
```

Two jobs. **Decorrelation:** consecutive steps in an episode are nearly identical, so a batch of them is effectively one sample and gives a badly biased gradient; uniform sampling from 100k transitions gives a batch spanning many episodes. **Sample reuse:** each transition is trained on many times, which is what makes off-policy methods like TD3/SAC far more sample-efficient than PPO — and that matters when a sample means a simulator step, more still on real hardware.

Off-policy means the data can come from an *older, worse* policy and still be usable, because the Bellman equation holds for any transition regardless of what generated it.

The draw above is **with** replacement. The toolkit's `buffers/er.py` draws without, which is three orders of magnitude slower once the buffer fills — measured, with the numbers, in `./PUBLICATION_MATERIALS.md` § Failure modes, 2026-08-12. Worth knowing that the obvious choice here is also the fast one.

PyTorch mechanics worth noticing

- `with torch.no_grad():` **around the target**. The target is a constant for this update. Without it, autograd builds a graph through the target networks and gradients flow where they must not — a subtly wrong algorithm that still runs.
- `zero_grad()` **before every backward()**. Gradients *accumulate* in PyTorch. Omit it and each update uses the sum of all previous gradients. Silent, and catastrophic.
- `@torch.no_grad()` **on act() and soft_update()**. Inference and in-place parameter surgery; neither needs a graph.
- `register_buffer` for `action_scale / action_bias`. Non-trainable tensors that should still move with `.to(device)` and be saved in `state_dict()`. A plain `self.x = torch.tensor(...)` silently stays on the CPU when the model moves to a GPU.

Part 3 — Where the demo differs from the Keras reference, deliberately

	Reference (keras_td3.py)	This demo	Why
Critic optimizers	one shared, losses summed	separate per critic	Same gradients, but Adam's per-parameter moments belong to each critic. Standard TD3 keeps them separate.
Seeding	wall-clock <code>time.time_ns()</code> ; action space never seeded	all three RNGs from <code>--seed</code>	<code>CLAUDE.md</code> : fixed seeds, always. The reference cannot reproduce a run.
Keras warm-up calls	<code>model(tf.zeros(...))</code> to build variables	none	PyTorch declares layer shapes up front.
Target init	<code>set_weights(get_weights())</code>	<code>copy.deepcopy</code>	Idiomatic, and no dependency on the warm-up call above.
Config	<code>.cfg</code> JSON files	inline constants	This is a demo. The shipped agent uses <code>cfgs/torch_td3.cfg</code> .

Part 4 — How this played out in the real port

These notes were written before any agent code existed. Three things the port then taught, each with evidence in `../PUBLICATION_MATERIALS.md`:

- **A silent default can be a confound**. Keras `Dense` initializes Glorot-uniform, PyTorch `Linear` Kaiming-uniform — a factor of 1.7 on a 256-wide layer. Left alone, a "PyTorch vs TensorFlow" curve is partly an *initialization* comparison, and nothing in either file names the initializer. Now a `cfg` key. (§ Method choices, 2026-08-17)
- **A passing suite is weak evidence**. 47 tests passed first try; three defects were then injected into `torch_td3.py` one at a time to see which tests noticed — `minimum`→`maximum` caught by 3, `(1 - dones)`→`dones` by 5, the `tau` swap by 1. Report what a suite catches, not that it passes. (§ Method choices, 2026-08-19)
- **The result is mixed, and that is the point**. `TorchTD3-v0` beats the Keras arm on two environments, loses on one, ties on one. A clean sweep would more likely mean a changed hyperparameter than a better implementation. (§ Comparisons, 2026-08-18)

Open questions this material raises — what the ported TD3 is ultimately *for*, whether the SINDy and uncertainty variants get ported, SAC or DDPG next — are collected in `../team-discussion.md`, not here.

Appendix — PyTorch vs TensorFlow, the five contrasts that mattered

The port was Keras → PyTorch, so these are the differences that actually showed up in the diff.

1. Define-by-run vs define-then-run. TF1 built a symbolic graph, then fed data through it in a `Session`. PyTorch has no build step: you write ordinary Python on real data and the graph is recorded as a side effect, fresh each forward pass and discarded after. So a PyTorch model is just Python — `print()` a tensor mid-forward-pass, set a breakpoint inside the network, branch on data-dependent values. TF2 made eager the default and largely closed the gap, but the lineage shows in `@tf.function` on `train_critic` ("trace this once, then run it as a static graph"). Fast, and it is why TF stack traces are hard to read: the failure happens inside a traced graph, not in your Python. PyTorch's `torch.compile` is opt-in.

2. Gradients: a tape object vs a tensor attribute. TF declares the recording region and asks the tape; PyTorch always records and accumulates *into* the tensors.

```
with tf.GradientTape() as tape:                # TF: start recording
    critic_loss1 = self.mse_loss(self.critic_model(states, actions, training=True), q_targets)
    gradients = tape.gradient(critic_losses, vars)    # ask the tape
    self.critic_optimizer.apply_gradients(zip(gradients, vars))
```

```
optimizer.zero_grad()    # torch: clear last step's gradients – mandatory
loss.backward()          # walk the graph, fill every .grad
optimizer.step()         # apply them
```

`zero_grad()` is mandatory because `.backward()` *adds to* `.grad` rather than replacing it. The accumulation default is deliberate — it lets you split one large batch across several backward passes — but forgetting it is the most common beginner bug. Note also that computing and applying gradients are separate calls on separate objects; TF fuses them into `apply_gradients`.

3. Explicit device placement. TF places tensors on a GPU automatically when one exists; PyTorch makes you say `x.to(device)` and errors if you mix CPU and GPU tensors. More verbose, but you always know where data is — and it is why the same code runs on this CPU-only laptop and on the RTX 5080 box with `device` as a `cfg` key.

4. Weights: attributes vs flat lists. Keras hands weights back as a flat list of arrays, so the soft target update becomes list surgery (`[w * tau + tw * (1 - tau) for w, tw in zip(get_weights(), ...)]` then `set_weights`). PyTorch modules expose named parameters you iterate and mutate in place. Same math, different idiom — and exactly what CLAUDE.md means by "**port logic, not framework**": write the PyTorch version of the *idea*, don't reproduce Keras's list-of-arrays dance.

5. Layer shapes are declared, not inferred. Keras infers input size on first call, hence the reference's `self.actor_model(tf.zeros([1, self.num_states]))` warm-up lines. PyTorch requires `nn.Linear(in_features, out_features)` up front, so those warm-up calls — and the `set_weights` copies that depend on them — disappear in the port.

Easy to misread: Keras passes `training=True/False` per call, PyTorch sets it as module state via `model.train()` / `model.eval()`. For the plain MLPs here it changes nothing — but worth knowing *why* the `training=` arguments vanish rather than assuming something was dropped.

What this track does not cover

- **SAC** — replaces the deterministic actor with a squashed Gaussian plus entropy regularisation. A bigger change than DDPG, which is TD3 minus the three tricks and nearly free once TD3 exists.

- **PPO** — the on-policy algorithm the Isaac Lab track actually uses. Track 3.
- **Autograd internals** — what the graph looks like, and `detach()` in detail.
- **Vectorized envs and the buffer/driver rework they need** — Track 3, and `../strategy.md` Part 3.
- **The toolkit's own structure** — registry, config, ABC, the SINDy variants. Track 2.

Deeper reference: `_reference/TD3_Robotic_Arm_RL_Guide.pdf`.

Stop 3 of 5 — run the demo

(not included - run it separately)

Stop 3 — run `td3_pendulum_demo.py`

Not included in this packet: you are running it separately. It is one self-contained file, 333 lines, at `RL_DT/_docs/learn/td3_pendulum_demo.py`.

```
python td3_pendulum_demo.py           # ~60 episodes, should reach about -200
python td3_pendulum_demo.py --episodes 30 # shorter
python td3_pendulum_demo.py --seed 1    # different seed
python td3_pendulum_demo.py --eval-every 5 # evaluation cadence
```

Read Stop 2 against this running. The point of the demo is that the whole algorithm fits in one file you can read top to bottom, with the hyperparameters as inline constants instead of in a `cfg`.

Four things to watch for, each of which Stop 2 explains:

- The **avg-20 training reward** climbing past -200 somewhere around episode 40-50. If it is still near -1200 after 30 episodes, something is wrong.
- The **two critic losses** moving together but not identically - they are independent estimators, and that is the whole point of clipped double-Q.
- The **actor loss going more negative** as the policy improves. It is $-Q(s, \pi(s))$, so a falling number is a rising Q. It is not a loss in the usual sense and it does not converge to zero.
- **Every episode is 200 steps.** `terminated` is always `False` on Pendulum - the episode always truncates. A green run here does *not* validate termination handling, which is why the ported agent needed mutation testing.

The shipped agent is `jlab_opt_control/agents/torch_td3.py` in the toolkit clone, with its hyperparameters in `cfgs/torch_td3.cfg`. This demo is a teaching artifact and a smoke test, not the port.

Stop 4 of 5 — the task

RL_DT/_docs/learn/3-isaac-and-rl-games.md - 219 lines

Part 1 is reading; Part 2 is a day at the keyboard. Nothing in it has been run - no RTX GPU on the laptop, so its verify-while-reading items are checks, not findings.

Track 3 — Isaac Sim, Isaac Lab, and the existing rl_games door task

What you get: what the digital twin actually is, the Isaac vocabulary you need and no more, and a guided read of the preliminary door task that already exists. **Read it against:** `_reference/digital_twin_models-main/` — the USD scene under `AIRoboticsLab/` and the seven code files under `RL_CloudTesting/`. **Size:** ~210 lines. Part 1 reading; Part 2 is a day at the keyboard. **Prerequisites:** Track 1 for the Gymnasium contract. Track 2 is not required.

Correction to the source this was built from. The Jul-24 briefing guide states in its second paragraph that "there is **no code** in this package — it is 3D scene + physics data." That was wrong, and finding out was the most consequential discovery of 2026-08-03: `RL_CloudTesting/` contains a **working Isaac Lab door task with rl_games PPO**. We are adapting something, not starting from scratch. Everything in Part 2 below is read from that code.

Part 1 — The twin

What a digital twin is here

Not a 3D model — a **physics simulation**: geometry (meshes), physics (mass, inertia, collision shapes, friction, gravity), articulation (joints, drives, limits) and optionally sensors. Because it obeys physics, an RL agent can take millions of practice attempts safely.

The file format is **USD** (Universal Scene Description), Pixar's open scene format and Isaac Sim's native language. The property that matters: a large scene is **composed** from many small referenced files, which is exactly how this twin is built.

What it models: a **Clearpath Ridgeback** omnidirectional base carrying a **Universal Robots UR5e** 6-DOF arm with a **Robotiq 2F-85** parallel-jaw gripper, next to an **Opentrons Flex** liquid handler whose door the arm must open. Use case #1 in the brief.

The vocabulary you actually need

1. **Stage / prims / references** — a scene is a tree of prims at paths like `/World/Ridgeback/ur5e/...`.
2. **Articulation** — a jointed robot. You control it through an articulation view: set joint targets, read joint position and velocity.
3. **Physics step vs render step** — RL steps physics many times; rendering is optional and expensive. This is what `--headless` and `decimation` are about.
4. **ManagerBasedRLEnv vs DirectRLEnv** — Isaac Lab's two ways to define a task. Ours is manager-based.
5. **num_envs and domain randomization** — cloning the scene N times for parallel training.

That is the whole list. You do not need to learn USD authoring.

Requirements, honestly

Isaac Sim is **RTX-only** — it will not run on the Intel Arc laptop this project is largely written on. It needs an NVIDIA RTX GPU, 8 GB VRAM to load a scene, 16 GB+ to be comfortable, and 24 GB+ for large `num_envs`. Ubuntu 22.04 is the first-class platform; Windows 11 works for opening and inspecting a scene and for small experiments. The assets were collected from **Isaac Sim 5.1** (recorded in `.collect.mapping.json`, author `prat615`, project `LDRD_RoboticsAI`), so use 5.1 or newer to avoid schema mismatches.

Practical consequence for us: everything in Part 2 can be *read* anywhere; nothing in it can be *run* without the RTX 5080 workstation. See `../environment.md`.

The scene, file by file

Under `AIRoboticsLab/`:

File	What it is
<code>AIRoboticsLab.usd</code> (40 KB)	Top-level scene , references everything else. Open this one.
<code>experiment.usd</code>	A second top-level scene. Which is canonical is an open question.
<code>SubUSDs/AIRoboticsLab_1.usd</code> (59 MB)	The lab room. Heaviest visual asset.
<code>SubUSDs/OpenTronsFlex.usd</code> (28 MB)	The instrument. Contains the UpperDoor prim — the door.
<code>SubUSDs/RidgebackWithURGripper.usd</code>	The assembled robot (composition).
<code>SubUSDs/ur5e_base / _physics / _sensor / _robot_schema.usd</code>	The arm, split by concern.
<code>SubUSDs/Robotiq_2F_85_physics_mimic.usda</code>	Gripper physics with mimic joints — one driven <code>finger_joint</code> , the others geared to it. The recommended variant.
<code>SubUSDs/Robotiq_2F_85_physics_loop.usda</code>	The same coupling modelled as a closed 4-bar linkage. The alternative.
<code>Defeatured_2F_85_PAD_OPEN_*.usd</code>	Simplified collision geometry for fast, stable contact physics.
<code>wxai_follower*</code> , <code>wxai_leader_*</code>	A Trossen WidowX-AI bimanual teleoperation rig. Bonus asset — relevant only if we want demonstrations for imitation learning.

Learn the `_base / _physics / _sensor / _robot_schema` convention — it is the standard Isaac modular robot layout and it tells you where to look. To change a motor, edit `_physics`; to read joint names, open `_physics` or the assembled robot; to add a camera, edit `_sensor`.

The `finger_joint` is the single command that opens and closes the gripper: drive stiffness 3, damping 2e-4, max force 26 N·m, range 0–47°.

First hands-on

Open `AIRoboticsLab.usd` in the Isaac Sim GUI. In the Stage panel find `Ridgeback`, `ur5e`, `Robotiq_2F_85`, and `OpenTronsFlex` → `UpperDoor`. Press Play and watch the robot settle under gravity — if it explodes or sinks through the floor, that is a collision problem to report, not something to work around. Open the Property panel on the door joint and read its drive stiffness, damping and limits.

Four capabilities to confirm before anything else: **read joint and door state, command joints, step physics, reset to a randomized initial pose.**

Part 2 — The door task that already exists

Seven files in `_reference/digital_twin_models-main/RL_CloudTesting/`, 1,756 lines total. Everything below is **read from the code, not run** — I have no RTX GPU. Treat the numbers as verified and the behaviour as unverified.

File	Lines	What it is
<code>ur5e_door_open_env_cfg.py</code>	545	Scene, observations, actions, 20 reward terms, terminations, sim settings
<code>ur5e_door_open_mdp.py</code>	595	The reward/observation/termination functions themselves — 31 of them
<code>ur5e_door_open_task.py</code>	26	<code>gym.register</code> for Isaac-UR5e-DoorOpen-v0
<code>rl_games_ppo_cfg.yaml</code>	81	The PPO hyperparameters
<code>train_rl_games.py</code>	258	Training entry point
<code>play_rl_games.py</code>	243	Checkpoint playback
<code>ReadMe.txt</code>	8	The two commands, which <code>CLAUDE.md</code> § How to run reproduces

Read it in this order

1. `ur5e_door_open_task.py` (26 lines, five minutes). The whole registration: `gym.register(id="Isaac-UR5e-DoorOpen-v0", entry_point="isaacsim.envs.ManagerBasedRLEnv", kwargs={"env_cfg_entry_point": UR5eDoorOpenEnvCfg, "rl_games_cfg_entry_point": <yaml>})`. Note `disable_env_checker=True`. This is the same registry idea as Track 2's toolkit and as `gym.make` — Track 1's point that Isaac Lab implements the Gymnasium interface without being part of Gymnasium's collection is *this line* of code.

2. `UR5eDoorOpenEnvCfg.__post_init__`, at the bottom of the env cfg. The concrete task shape:

- **Actions** — `JointPositionActionCfg` on the 6 arm joints (`scale=0.5`, `use_default_offset=True`) plus `BinaryJointVelocityActionCfg` on the gripper (open `+1.0`, close `-1.0`). So the policy commands joint position targets, not torques, and the gripper is binary rather than continuous. **The Ridgeback base is not in the action space.**
- **Observations** (`concatenate_terms=True`, `enable_corruption=True`): arm `joint_pos_rel`, `joint_vel_rel`, door joint position and velocity, `rel_ee_handle_distance`, `last_action`, and a flag-gated `door_open_progress`. Privileged simulator state throughout — **no vision, no force, no torque, no contact flags.**
- **Timing** — `sim.dt = 1/60`, `decimation = 2`, `episode_length_s = 20.0`. So the policy acts at 30 Hz for **600 steps per episode.**
- **Scene** — `num_envs=512`, `env_spacing=2.5`, with `FrameTransformerCfg` supplying the end-effector and handle frames the distance rewards need.

3. `RewardsCfg` — 20 terms, and read the weights. They form a scripted sequence: approach the handle (1.2), approach from the robot side (1.5), align (1.0), pre-grasp orientation (2.0), approach the gripper (4.0), align the grasp around the handle (2.0), grasp (5.0), gated door-open fraction (2.0 and 2.5), multi-stage open (2.5), ungrasp after open (1.5), return home (2.0), sequence completion bonus (2.0), and two penalties — moving the door while ungrasped (`-6.0`) and levering under the door (`-4.5`). Then six `script_guided_*` terms: a phase gate, a grasp-phase bonus (1.2), arc handle tracking (1.4), a grip-slip penalty (`-2.5`), a base `dof0` profile reward (0.4), and trajectory tracking (2.0). Finally two generic regularizers: `action_rate_l2` (`-0.005`) and `joint_vel_l2` (`-0.001`).

4. `ur5e_door_open_mdp.py` — the 31 functions behind those terms. Read `door_open_fraction`, `grasp_state_gate`, `sequence_completion_bonus` and `script_guided_trajectory_tracking_reward` (with its `_interp_piecewise` helper), in that order. This file is where the task design actually lives.
5. `TerminationsCfg` — two terms: `time_out` (`time_out=True`, so it surfaces as **truncated**) and `sequence_complete` (a real **terminated**). Good news for Track 1's distinction: unlike Pendulum, this task genuinely terminates, so the `(1 - done)` factor is live code here and getting it wrong would matter.
6. `rl_games_ppo_cfg.yaml` — `a2c_continuous` / `continuous_a2c_logstd`, an actor-critic MLP [256, 128, 64] with `elu`, **shared trunk** (`separate: False`), `fixed_sigma: True`. Config: `gamma 0.99`, `tau 0.95` (this is GAE- λ , *not* the Polyak τ from Track 1 — same letter, different thing), `learning_rate 1e-3` with `lr_schedule: adaptive` and `kl_threshold 0.01`, `e_clip 0.2`, `entropy_coef 0.01`, `horizon_length 32`, `minibatch_size 32768`, `mini_epochs 5`, `normalize_input/normalize_value/value_bootstrap` all `true`, `max_epochs 2000`, `seed 42`.

Two things to notice, because they connect to what you already know. `value_bootstrap: True` is `rl_games` doing exactly what Track 1 says must happen at a time limit — bootstrapping the value function through truncation instead of zeroing it. And `seed: 42` is fixed here, which the reference toolkit never manages.

Four things to verify while reading — with what I suspect

I have not run this, so these are **checks, not findings**. Confirm each before building on the task.

1. **The batch/minibatch arithmetic.** `rl_games` builds a batch of `horizon_length × num_actors`, where `num_actors: -1` resolves to `num_envs`. The ReadMe's command uses `--num_envs 256`, giving $32 \times 256 = 8,192$, against a `minibatch_size` of **32,768** — four times larger than the batch. The `cfg`'s own `num_envs=512` gives 16,384, still half. `num_envs = 1024` is the value that makes it come out exactly. Check whether `rl_games` errors, silently clamps, or the ReadMe command was never the one actually used.
2. **Zero domain randomization.** `EventCfg` has one term, `reset_joints_by_offset`, with `position_range: (0.0, 0.0)` and `velocity_range: (0.0, 0.0)`. Every episode starts from the identical pose, so **nothing in the brief's "variable geometry" requirement is exercised yet** and a trained policy has no reason to generalize. This is the single largest gap between this task and the project's stated goal.
3. **A reward term for a joint the policy cannot command.** `script_guided_base_dof0_profile_reward` (weight 0.4) shapes a base degree of freedom, but the action space contains only the 6 arm joints and the gripper. Either the base is actuated somewhere I have not found, or this term rewards something the policy has no way to affect. Verify against a live run.
4. **No force or torque anywhere.** Not in the observations, not in the rewards, no `ContactSensor` in the scene `cfg` — only `FrameTransformerCfg`. The brief explicitly requires "no brute-force pushing", so a force/torque penalty is a *missing requirement*, not a refinement. It is on the roadmap in `CLAUDE.md` and it is why the tactile-data question matters in `../team-discussion.md`.

And the design judgement to form your own view on: **the reward is heavily script-guided**. Six terms track a reference trajectory and several more gate on sequence phase. That will very likely produce a door-opening motion, and it is close to imitating a scripted plan under a policy's name. Reducing that dependence is a medium-term roadmap item; decide for yourself where the line sits between shaping and telling.

What is genuinely undocumented

Nothing in this project explains **`rl_games` itself** — its `a2c_continuous` implementation, how `horizon_length` and `mini_epochs` interact, how it wraps a vectorized Isaac Lab env, or what its checkpoint format is. `train_rl_games.py` and `play_rl_games.py` (258 + 243 lines) are the local entry points and worth reading for the wrapper boilerplate,

but the library is upstream. Read it at its own repo when you need it. This is a real gap, stated rather than papered over.

What this track does not cover

- **PPO as an algorithm.** Track 1 covers off-policy TD3; PPO is on-policy, with a clipped surrogate objective and GAE, and nothing here teaches it. If you only read one thing, read why `e_clip` and `entropy_coef` exist.
- **Installing Isaac Sim / Isaac Lab** — needs the workstation; see `../environment.md`.
- **Vectorized envs on the agent side** — how a 512-env batched GPU tensor changes the driver and replay buffer: `../strategy.md`.
- **The task's assignment and research framing** — Track 4.
- **The open questions** these files raise: `../team-discussion.md`.

Stop 5 of 5 — the codebase

RL_DT/_docs/learn/2-toolkit.md - 182 lines

Read this when you next touch toolkit code. Use the reading order in its section 8 over a day at the keyboard.

Track 2 — The SciOptControlToolkit

What you get: the map you need to find your way around `jlab_opt_control` — what each layer does, the one pattern that explains all of it, and where our own code sits inside it. **Read it against:** the live clone at `<PROJECT_ROOT>/SciOptControlToolkit`, branch `develop-dw`. **Size:** ~180 lines. One sitting, then use the reading order in § 8 over a day at the keyboard. **Prerequisites:** Track 1 — this track assumes you know what TD3 is.

Merged from a pre-code walkthrough and the Jul-24 briefing guide, then corrected against the installed clone. Measured numbers and code defects are **not** repeated here — they live in `../PUBLICATION_MATERIALS.md`.

1. What it is

`jlab_opt_control` is Malachi Schram's (JLab) TensorFlow/Keras toolkit for RL-based control of **particle accelerators** — distributing RF gradients across CEBAF's superconducting cavities to minimize cryogenic heat load and trip rate under a hard total-energy constraint. About 4,500 lines.

It is not a robotics framework and was never meant to be. We use it because it is where our group's method contribution already lives, and because it is Malachi's code — the patterns in it are the patterns our work is expected to fit.

One pattern explains the whole repo: registry + config. Every component — agent, model, buffer, env — is registered by a string ID in a subpackage's `__init__.py` (`register(id="KerasTD3-v0", entry_point=..., kwargs={"cfg": "keras_td3.cfg"})`) and built by `make("KerasTD3-v0", ...)`. Deliberately modelled on `gym.make()`.

Hyperparameters never appear in code; they live in `cfgs/*.cfg` as JSON. Internalize this and the repo stops being surprising: an experiment is assembled from the command line, and adding a capability means adding a file plus one registry line.

2. The two papers, mapped onto the code

Both from this group, both in `_reference/`. Reading the code without them tells you *what* it does and not *why*.

Paper	Idea	Where in code
Rajput 2025 (MLST 6 025018)	Compares MOGA, MOBO, CMO-TD3, CMO-DDRL for multi-objective CEBAF optimization. Differentiable-sim RL wins at high dimension (200 cavities); plain TD3 fails the energy constraint there.	The TD3 baseline in <code>agents/keras_td3.py</code> is the CMO-TD3 foundation. The CEBAF surrogate envs are in the separate PACES package, optionally imported by <code>run_continuous.py</code> .
Colen 2026 (MLST 7 015005)	LC-TD3 / Sparse-LC-TD3: augment TD3 with a <i>learnable</i> physics surrogate (neural or SINDy sparse dictionary) that predicts an observable and constrains the policy. The sparse version is interpretable — it recovers $E = \sum G_i l_i$ — and is the only method that converges on the 197-cavity linac without a differentiable env.	<code>agents/keras_sindy_critic_td3.py</code> , <code>keras_sindy_uncertainty_td3.py</code> , <code>core/sindy_lib_core.py</code> , <code>utils/sindy_lib/</code> , <code>models/sindy_network.py</code> , <code>models/uqsindy_network.py</code>

The transferable lesson, and the reason this toolkit is in our project at all: **injecting known physics is what makes RL scale on high-dimensional, hard-constrained systems**. Isaac Lab supplies the parallel-physics side; this toolkit supplies the agent-side patterns.

3. Repo map

```
jlab_opt_control/
├── core/          abstract base classes: Agent, Model, ReplayBuffer, SINDyLibrary
├── agents/        TD3 (main), DDPG, SAC, REDQ-TD3, Uncertainty-TD3, SINDy-TD3 variants
│                   + torch_td3.py ← ours
├── models/        actor/critic MLPs built from cfg, SINDy networks
│                   + torch_models.py, core/torch_model_core.py ← ours
├── buffers/       er.py (uniform replay), per.py (prioritized replay)
├── envs/          Circle2D toy env + a gym-style registry
│                   + robotics_flat.py ← ours
├── cfgs/          JSON – ALL hyperparameters
│                   + torch_td3.cfg ← ours
├── drivers/       run_continuous.py – THE entry point
└── utils/         cfg parsing, git-hash logging, SINDy feature libraries
```

Layer	Files	What to know
Driver	drivers/run_continuous.py	The whole RL loop. <code>main()</code> → <code>run_opt()</code> → <code>run_episode()</code> . ~390 lines; the ~50 that matter are the episode loop.
Agent	agents/keras_td3.py and variants	TD3 = DDPG + three fixes, all visible in <code>train_critic()</code> / <code>train_actor()</code> .
Models	models/actor_fcnn.py, critic_fcnn.py	Plain MLPs. Actor 2×256 ReLU → tanh scaled to action bounds; critic $\text{concat}(s,a) \rightarrow 2 \times 256 \rightarrow \text{scalar } Q$.
Buffers	buffers/er.py, per.py	Uniform vs prioritized replay.
Envs	envs/circle_env.py	Circle2D-v0 (see § 6). Standard Gymnasium envs work directly.
Registry	*/registration.py	The <code>make()</code> / <code>register()</code> machinery.
Configs	cfgs/*.cfg	keras_td3.cfg: lr $3e-4$ both nets, γ 0.99, τ 0.005, batch 256, warmup 2500, ER buffer 1M.

4. Execution flow — start here

```
python -m jlab_opt_control.drivers.run_continuous \
    --agent KerasTD3-v0 --env Pendulum-v1 --nepisodes 100 --nsteps 200
```

1. `main()` parses args → `run_opt()`.
2. `create_and_configure_env()` tries three registries in order: standard Gymnasium, the toolkit's own envs (Circle2D-v0), then PACES (optional, absent here).
3. The env is wrapped in `TimeLimit`; the agent is built via `agents.make()`.
4. `run_episode()`: `agent.action(state)` → `env.step(action)` → `agent.memory((s,a,r,s',terminate))` → `agent.train()`. **One gradient update per env step**, single environment.
5. Every `inference_interval` (10) episodes a noise-free evaluation episode runs, and the model is checkpointed when the rolling average inference reward improves by `model_save_threshold`.

6. Everything needed to reproduce a run lands in `./results/index###_env..._time.../` — TensorBoard scalars, `results.npy`, buffer snapshots, config copies, and the git SHA.

Two things in that list are **measurably broken** — the checkpoint criterion never rejects anything, and the evaluation runs on the training env object. Do not take steps 5–6 at face value; read `../PUBLICATION_MATERIALS.md` § Failure modes before you trust a number that came out of this driver.

Reading tip that will save you an hour. Derive the agent interface from the driver and the tests, **not** from the ABC. `core/agent_core.py` omits `memory` and `buffer`, both of which `run_continuous.py` requires — the declared interface is narrower than the real one.

5. The TD3 agent is the extension point

Read `agents/keras_td3.py` closely, because it is the class every research variant is a delta from. Six networks (actor + target, two critics + two targets), built via `models.make()` from `cfg`. `train_critic()` holds tricks 1 and 2, `train_actor()` holds trick 3, and `action()` handles the warm-up gate and exploration noise.

The structural finding that shaped our whole plan: **KerasSINDyCriticTD3**, **KerasUncertaintyTD3** and **KerasSINDyUncertaintyTD3** all subclass the TD3 agent and override `train_critic`, changing almost nothing else. So the SINDy-structured critic and the uncertainty-aware critic are not separate algorithms — they are modifications of the Bellman-target computation inside TD3. A clean overridable `train_critic` is therefore the single most important design property of any port, which is why our port did TD3 first. Detail: `../strategy.md`.

6. Envs — and the assumption they reveal

`Circle2D-v0` has a 2D state and 2D action, reward $1000 \cdot \exp(-5 \cdot \|s\| - 0.95|)$ peaking on a circle of radius 0.95, and a **single-step episode** by default. It is an *optimization problem dressed as RL*, which mirrors exactly how the papers treat CEBAF: pick the cavity gradients once, collect the reward.

Internalize this. The toolkit's home problem is near-single-step optimization under a hard constraint. The door task is a genuine long-horizon sequential MDP — approach, grasp, swing, over hundreds of timesteps. That single difference drives most of what has to change: reward shaping, termination design, algorithm choice, and parallelism.

7. The physics-informed agents — the research crown jewel

`core/sindy_lib_core.py` and `utils/sindy_lib/` implement SINDy-style feature libraries (Polynomial, Fourier) composable with `+` (concat) and `*` (product), mapping `[s, a]` to candidate terms $(1, G_i, G_i G_j, G_i^2, \dots)$.

`KerasSINDyCriticTD3` then adds two things to TD3: a **SINDy critic**, a sparse-regularized linear model over those library features trained each step to match $\min(Q_1, Q_2)$ of the neural critics; and a modified actor loss $q = \beta \cdot q_{\text{sindy}} + (1-\beta) \cdot q_{\text{critic}}$, where `cfg.sindy_beta` defaults to **1.0** — meaning the actor trains *entirely* through the interpretable surrogate. Because the surrogate is a linear combination of known basis functions, you can read the learned equation off and check it against physics.

`KerasUncertaintyTD3` has critics output $(Q, \log \sigma^2)$ with a KL regularizer on the variance head, and during training picks the action with **maximum critic uncertainty** among 256 random candidates — active-learning-style exploration.

One discrepancy worth carrying to a meeting. Colen 2026's LC-TD3 surrogate predicts a physical *observable* (energy) with an explicit constraint penalty. The released SINDy agent instead distills the *critic* into a sparse model. Same machinery, different target — the observable-constraint variant presumably lives with the PACES envs, which we do not have. This is an open question, not a settled reading: `../team-discussion.md`.

8. Reading order

At the keyboard, in the live clone. Items 3–4 are ours and are the fastest way to see the patterns above applied by someone who had to learn them recently.

1. `drivers/run_continuous.py` — `run_episode()` then `run_opt()`. The whole loop.
2. `agents/keras_td3.py` — `__init__`, `action()`, `train()`, `train_critic()`, `train_actor()`.
3. `agents/torch_td3.py` — the same algorithm in PyTorch, ours. Diff it against #2 mentally.
4. `utests/test_torch_td3.py` — 47 tests. Reading tests is the fastest way to learn what the contract actually is, including the parts the ABC omits.
5. `models/actor_fcnn.py` + `critic_fcnn.py`, then `buffers/er.py` → `per.py`.
6. `envs/circle_env.py` plus any `registration.py` — the registry pattern in ten lines.
7. Colen 2026 § 3, then `agents/keras_sindy_critic_td3.py` → `core/sindy_lib_core.py`.
8. `notebooks/ExaminePendulumPolicy.ipynb` — loads a trained checkpoint and plots the policy. The useful one; three of the other four need `pysindy` (installed nowhere) and `ExaminePACESPolicy.ipynb` hardcodes a path on another developer's Mac.

The runnable smoke test and the baseline number to expect are in `CLAUDE.md` § How to run — one command, already verified on this machine. Don't take the install instructions from the old briefing docs; `env.yaml` pins Python 3.13, and the whole dependency set in fact resolves on 3.11.9.

9. What this track does not cover

- **The port's scope and status** — what was ported, what was reused, what was deliberately deferred, and the architecture options: `../strategy.md`.
- **The measured defects** and every number with evidence attached: `../PUBLICATION_MATERIALS.md`.
- **PER and REDQ-TD3** — registered and untested by us end to end.
- **The PACES envs** — a separate package we do not have.
- **Isaac Lab and rl_games** — Track 3.

Deeper reference: `_reference/SciOptControlToolkit-wiki`, `_reference/SciOptControlToolkit_Walkthrough.pdf`, and the two MLST papers `_reference/Rajput_2025_*.pdf` and `_reference/Colen_2026_*.pdf`.